

SweetNavBar

A custom floating bottom navigation bar with smooth icon switching, optional labels, rounded corners, shadows, and **fixed height support**.

✨ Features

- Fixed height navigation bar
- Active / inactive icon support
- Optional labels per item
- Custom background color
- Custom border radius
- Custom box shadow
- SafeArea aware (bottom)

🐘 Constructor

```
SweetNavBar({  
  required int currentIndex,  
  required List<SweetNavBarItem> items,  
  required double height,  
  ValueChanged<int>? onTap,  
  double borderRadius = 30,  
  Color backgroundColor = Colors.white,  
  List<BoxShadow>? boxShadow,  
})
```

🦊 Parameters

Parameter	Type	Required	Description
<code>currentIndex</code>	<code>int</code>	✓	Currently selected index
<code>items</code>	<code>List<SweetNavBarItem></code>	✓	Navigation items
<code>height</code>	<code>double</code>	✓	Fixed height of the navbar
<code>onTap</code>	<code>ValueChanged<int></code>	✗	Callback on item tap
<code>borderRadius</code>	<code>double</code>	✗	Corner radius

Parameter	Type	Required	Description
backgroundColor	Color	✗	Background color
boxShadow	List<BoxShadow>	✗	Shadow configuration

SweetNavBarItem

```
class SweetNavBarItem {
  final Widget icon;
  final Widget? activeIcon;
  final String? label;

  const SweetNavBarItem({
    required this.icon,
    this.activeIcon,
    this.label,
  });
}
```

Example Usage

```
SweetNavBar(
  currentIndex: _selectedIndex,
  height: 70,
  borderRadius: 40,
  backgroundColor: Colors.white,
  onTap: (index) {
    setState(() => _selectedIndex = index);
  },
  items: const [
    SweetNavBarItem(icon: Icon(Icons.home), label: 'Home'),
    SweetNavBarItem(icon: Icon(Icons.article), label: 'Resume'),
    SweetNavBarItem(icon: Icon(Icons.layers), label: 'Templates'),
    SweetNavBarItem(icon: Icon(Icons.settings), label: 'Settings'),
  ],
)
```



Full Source Code

```
import 'package:isaacmwesigw352/constants/common_imports.dart';

class SweetNavBar extends StatelessWidget {
  final int currentIndex;
  final ValueChanged<int>? onTap;
  final List<SweetNavBarItem> items;

  final double height;
  final double borderRadius;
  final Color backgroundColor;
  final List<BoxShadow>? boxShadow;

  const SweetNavBar({
    super.key,
    required this.currentIndex,
    required this.items,
    required this.height,
    this.onTap,
    this.borderRadius = 30,
    this.backgroundColor = Colors.white,
    this.boxShadow,
  });

  @override
  Widget build(BuildContext context) {
    return SafeArea(
      top: false,
      bottom: true,
      child: Container(
        height: height,
        padding: const EdgeInsets.symmetric(horizontal: 16),
        decoration: BoxDecoration(
          color: backgroundColor,
          borderRadius: BorderRadius.circular(borderRadius),
          boxShadow: boxShadow ??
            [
              BoxShadow(
                color: Colors.black.withAlpha(20),
                blurRadius: 10,
                offset: const Offset(0, 5),
              ),
            ],
        ),
        child: Row(
```

```

mainAxisAlignment: MainAxisAlignment.spaceAround,
children: List.generate(items.length, (index) {
  final item = items[index];
  final isSelected = index == currentIndex;

  return InkWell(
    onTap: () => onTap?.call(index),
    borderRadius: BorderRadius.circular(24),
    child: Column(
      mainAxisAlignment: MainAxisAlignment.center,
      mainAxisSize: MainAxisSize.min,
      children: [
        SizedBox(
          height: 28,
          width: 28,
          child: isSelected
            ? item.activeIcon ?? item.icon
            : item.icon,
        ),
        if (item.label != null) ...[
          const SizedBox(height: 4),
          Text(
            item.label!,
            style: TextStyle(
              fontSize: 12,
              fontWeight:
                isSelected ? FontWeight.w600 : FontWeight.normal,
              color: isSelected ? Colors.blue : Colors.grey,
            ),
          ),
        ],
      ],
    ),
  );
});
},
),
);
}
}

class SweetNavBarItem {
  final Widget icon;
  final Widget? activeIcon;
  final String? label;

  const SweetNavBarItem({
    required this.icon,

```

```
        this.activeIcon,  
        this.label,  
    });  
}
```



Notes

- `height` is **mandatory** and enforced
- The bar remains floating-safe with system navigation bars
- Works perfectly with `Stack + Align.bottomCenter`

If you want **animated indicator**, **bubble effect**, or **glassmorphism**, say the word 